Complete CI/CD Pipeline Using GitHub, Jenkins, Docker, and EC2

Introduction

Continuous Integration and Continuous Deployment (CI/CD) is a modern software development practice that automates the process of building, testing, and deploying applications.

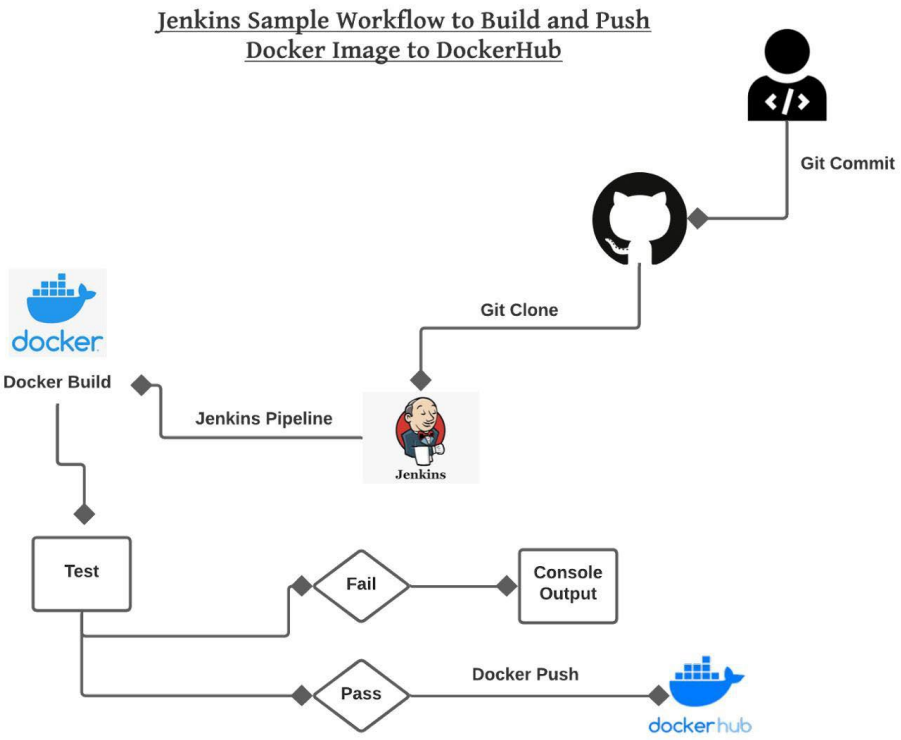
In this project, a complete CI/CD pipeline was implemented using:

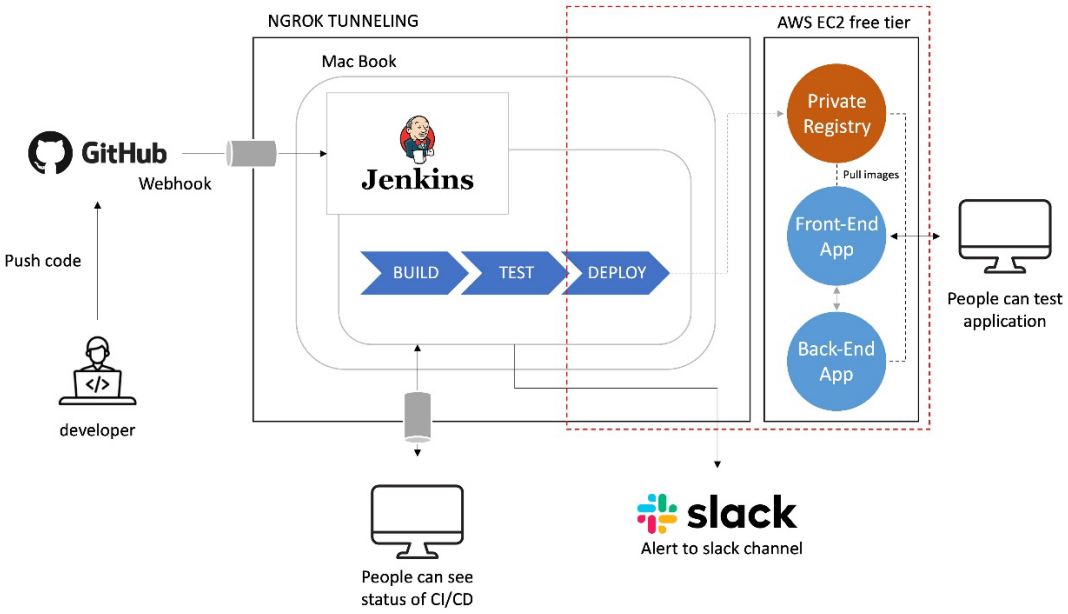
- GitHub
- Jenkins
- Docker
- Amazon Web Services EC2

The pipeline automatically deploys website updates whenever code is pushed to GitHub.

Project Architecture

CI/CD Workflow Architecture





Architecture Flow

Developer
↓
GitHub Repository
(Webhook Trigger)
↓
Jenkins Server (EC2)
↓
Build Docker Image
↓
Stop Old Container
↓
Run New Container
↓
Website Live on Port 9090

Technologies Used

TECHNOLOGY	PURPOSE
GITHUB	Source code management
JENKINS	CI/CD automation
DOCKER	Containerization
AMAZON WEB SERVICES EC2	Cloud server hosting
NGINX	Web server

Step 1 — Launch AWS EC2 Instance

Create EC2 Instance

Launch an Ubuntu EC2 instance from AWS Console.

Recommended configuration:

SETTING	VALUE
OS	Ubuntu Server 24.04
INSTANCE TYPE	t2.micro
STORAGE	20 GB
SECURITY GROUP	Allow 22, 8080, 9090

Configure Security Group

Open these ports:

PORT	PURPOSE
22	SSH
8080	Jenkins
9090	Website

Step 2 — Install Docker

Update Packages

```
sudo apt update
```

Install Docker

```
sudo apt install docker.io -y
```

Start Docker

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Add Jenkins User to Docker Group

```
sudo usermod -aG docker jenkins
```

```
sudo usermod -aG docker ubuntu
```

Step 3 — Install Jenkins

Install Java

```
sudo apt install openjdk-17-jdk -y
```

Add Jenkins Repository

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Install Jenkins

```
sudo apt update
sudo apt install jenkins -y
```

Start Jenkins

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Access Jenkins

Open browser:
`http://EC2-PUBLIC-IP:8080`

Example:
http://51.20.91.105:8080

Unlock Jenkins

Get initial password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Paste password into Jenkins UI.

Install Jenkins Plugins

Install recommended plugins.
Important plugins:

- Git Plugin
- GitHub Integration Plugin
- Pipeline Plugin
- Docker Pipeline Plugin

Step 4 — Create GitHub Repository

Create Repository

Example repository: . it should contain your code to be displayed

GitHub Repository

Step 5 — Create Dockerfile

Dockerfile: Create a file name "Dockerfile" inside the repot and insert the below code

```
FROM nginx:alpine

COPY . /usr/share/nginx/html

EXPOSE 80
```

Explanation:

- `FROM nginx:alpine` : Uses lightweight Nginx image.
- `COPY . /usr/share/nginx/html` Copies website files into Nginx web directory.
- `EXPOSE 80` Exposes web server port inside container.

Step 6 — Create Jenkinsfile

Jenkins Pipeline Script

Create a file name "Jenkinsfile" inside the repot and insert the below code

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                git 'https://github.com/Aswinvtk/aswinvtk.github.io.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t aswinvtk/github-pages-site .'
            }
        }

        stage('Stop Old Container') {
            steps {
                sh 'docker rm -f github-pages-site || true'
            }
```

```
        }

        stage('Run Container') {
            steps {
                sh '''
                docker run -d \
                --name github-pages-site \
                -p 9090:80 \
                aswinvtk/github-pages-site
                '''
            }
        }
    }
}
```

Jenkins Pipeline Explanation

`git 'repository-url'` Clones latest code from GitHub.

`docker build -t image-name .` Creates Docker image from Dockerfile.

`docker rm -f container-name` Removes existing running container.

`docker run -d -p 9090:80` Starts new container.Port mapping:

EC2 Port	Container Port
9090	80

Step 7 — Create Jenkins Pipeline Job

Create New Item

In Jenkins:

`New Item → Pipeline`

Job name:

`bio`

Configure Pipeline

Under Pipeline:

Select:

`Pipeline script from SCM`

SCM:

```
Git
```

Repository URL:

```
https://github.com/Aswinvtk/aswinvtk.github.io.git
```

Branch:

```
*/main
```

Script Path:

```
Jenkinsfile
```

Step 8 — Configure GitHub Webhook

GitHub Webhook Settings

Go to:

```
Repository → Settings → Webhooks
```

Add webhook.

Payload URL

```
http://51.20.91.105:8080/github-webhook/
```

Content Type

```
application/json
```

Event

```
Just the push event
```

Step 9 — Enable Jenkins Build Trigger

Inside Jenkins Job:

```
Configure → Build Triggers
```

Enable:

```
GitHub hook trigger for GITScm polling
```

This automatically triggers builds after GitHub push.

Step 10 — Push Code to GitHub

Git Commands

```
git add .
git commit -m "Updated website"
git push origin main
```

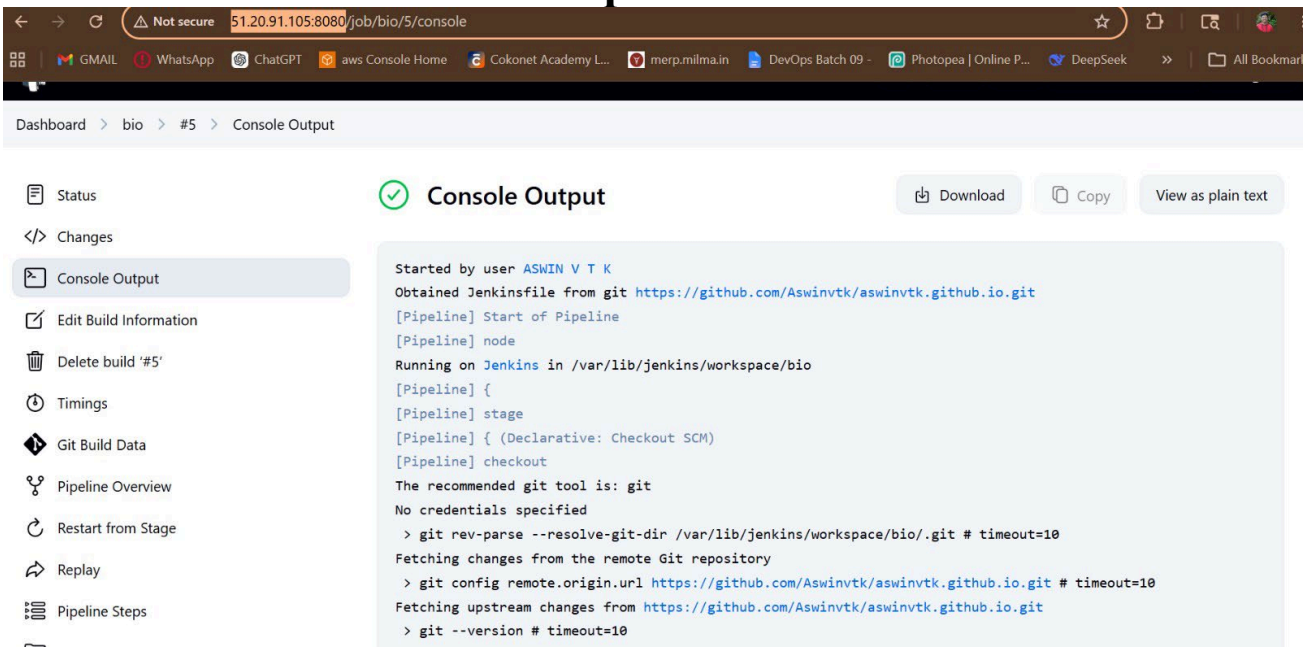
Step 14 — Jenkins Build Process

Jenkins Automatically:

1. Receives webhook
2. Pulls latest code
3. Builds Docker image
4. Stops old container
5. Deploys new container

Jenkins Console Output

Successful Build console Output



Example console log:

```
Started by GitHub push

[Pipeline] stage
[Pipeline] { (Build Docker Image)

+ docker build -t aswinvtk/github-pages-site .

Successfully built 7f47c6938817
Successfully tagged aswinvtk/github-pages-site:latest

[Pipeline] stage
[Pipeline] { (Run Container)

+ docker run -d --name github-pages-site -p 9090:80

Finished: SUCCESS
```

Docker Image Verification

Check Images

```
docker images
```

Output:

```
REPOSITORY                      TAG       IMAGE ID
aswinvtk/github-pages-site      latest    7f47c6938817
nginx                           alpine    2f07d83bf561
```

Running Container Verification

Check Running Containers

```
docker ps
```

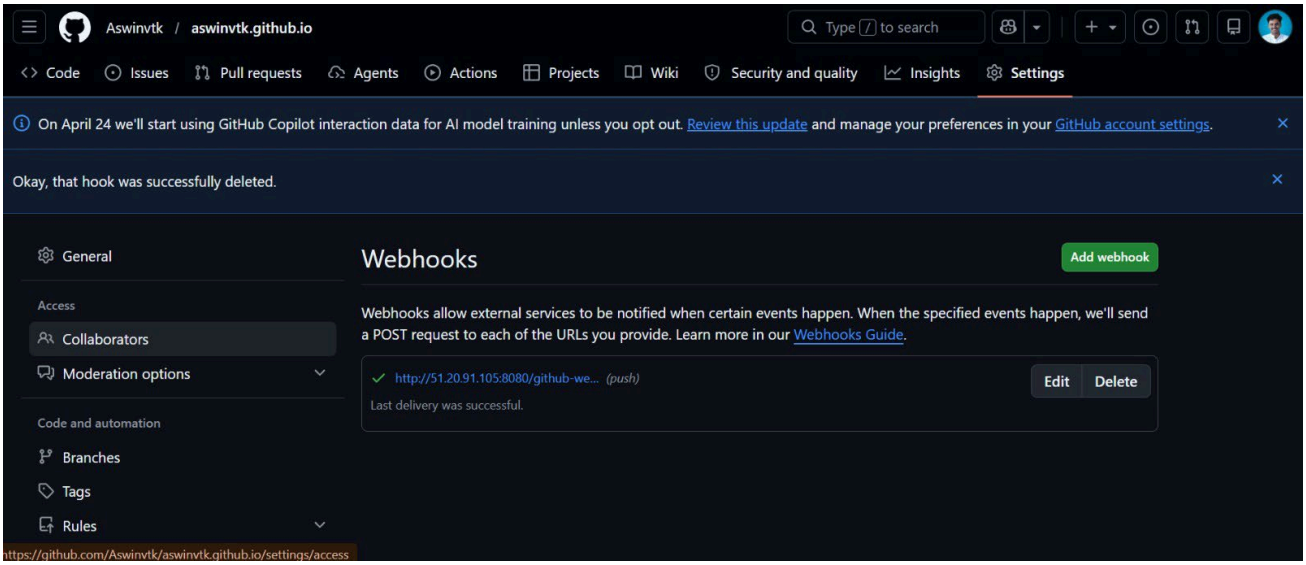
Output:

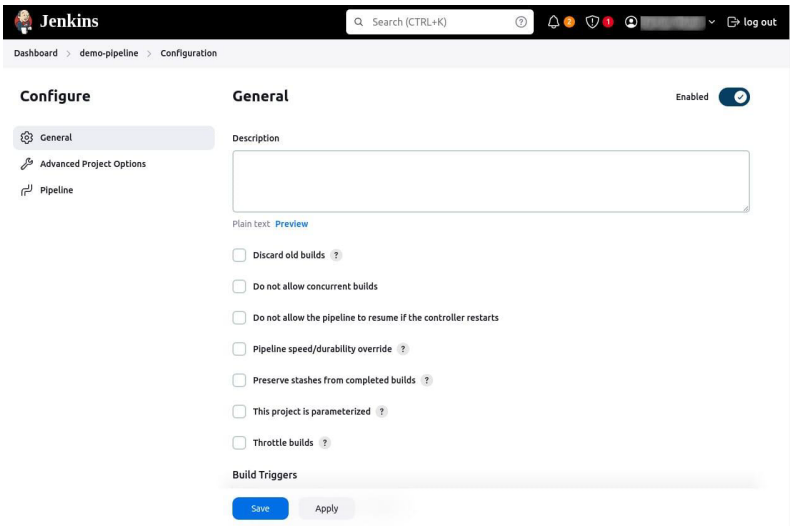
```
CONTAINER ID   IMAGE                          PORTS
ea50f8ccbaea   aswinvtk/github-pages-site      0.0.0.0:9090->80/tcp
```

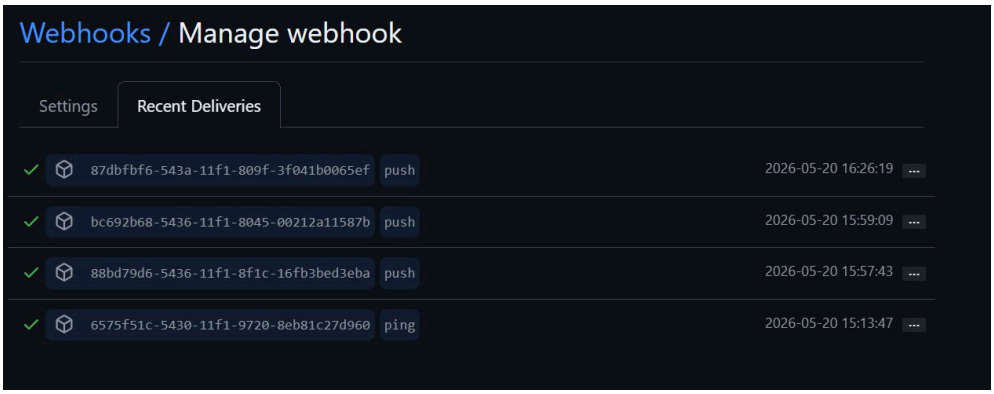
Access Website

Open browser:

```
http://51.20.91.105:9090
```


Aswinvtk / aswinvtk.github.io
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.
Okay, that hook was successfully deleted.
General
Access
Collaborators
Moderation options
Code and automation
Branches
Tags
Rules
Webhooks
Add webhook
Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our Webhooks Guide.
http://51.20.91.105:8080/github-we... (push)
Last delivery was successful.
Edit
Delete
https://github.com/Aswinvtk/aswinvtk.github.io/settings/access


Jenkins
Search (CTRL+K)
log out
Dashboard
demo-pipeline
Configuration
Configure
General
Advanced Project Options
Pipeline
General
Enabled
Description
Plain text Preview
Discard old builds
Do not allow concurrent builds
Do not allow the pipeline to resume if the controller restarts
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Throttle builds
Build Triggers
Save
Apply


Webhooks / Manage webhook
Settings
Recent Deliveries
87dbfbf6-543a-11f1-809f-3f041b0065ef
push
2026-05-20 16:26:19
bc692b68-5436-11f1-8045-00212a11587b
push
2026-05-20 15:59:09
88bd79d6-5436-11f1-8f1c-16fb3bed3eba
push
2026-05-20 15:57:43
6575f51c-5430-11f1-9720-8eb81c27d960
ping
2026-05-20 15:13:47

Challenges Faced

Common Issues

Problem	Solution
Docker permission denied	Add Jenkins user to docker group
Webhook not triggering	Enable GitHub hook trigger
Port inaccessible	Open EC2 security group ports
Jenkins build fails	Verify Docker service

Conclusion

- A CI/CD pipeline was implemented using GitHub, Jenkins, Docker, and AWS EC2.
- First, an EC2 Ubuntu server was created and required tools such as Java, Jenkins, Docker, and Git were installed.
- The website source code, along with the Dockerfile and Jenkinsfile, was uploaded to a GitHub repository.
- Jenkins was then connected to the GitHub repository using a pipeline job.
- A GitHub webhook was configured to automatically trigger Jenkins whenever code was pushed to the repository.
- After each push, Jenkins automatically pulled the latest code, built a Docker image, removed the old container, and deployed a new container on the EC2 server.
- This enabled automatic deployment of the updated website without manual intervention, achieving a complete CI/CD workflow.

This project successfully implemented a fully automated CI/CD pipeline using:

- GitHub
- Jenkins
- Docker
- Amazon Web Services EC2

The pipeline automatically deploys website updates whenever code is pushed to GitHub, reducing manual effort and improving deployment speed and reliability.

This architecture demonstrates a practical DevOps implementation suitable for modern web application deployment.

Here’s a simple pocket-note style CI/CD implementation guide using GitHub + Jenkins + Docker + Amazon Web Services EC2.

 # CI/CD Quick Notes

Architecture

```text id="y4x30v"

GitHub

↓

Webhook

↓

Jenkins

↓

Docker Build

↓

Docker Run

↓

Website Live

```

 # 1. Launch EC2

Open ports:

```text id="h6htmx"

22 → SSH

8080 → Jenkins

9090 → Website

```

 # 2. Install Docker

```bash id="20wrm4"

sudo apt update

sudo apt install docker.io -y

sudo systemctl start docker

sudo systemctl enable docker

```

Add users:

```bash id="m6xv5z"

sudo usermod -aG docker ubuntu

sudo usermod -aG docker jenkins

```

 # 3. Install Jenkins

Install Java:

```bash id="d29o5x"

sudo apt install openjdk-17-jdk -y

```

Install Jenkins:

```bash id="r88b3f"
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```bash id="v0m1l4"
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```bash id="k0w9o0"
sudo apt update
sudo apt install jenkins -y
```

Start Jenkins:

```bash id="g9c4h9"
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

 # 4. Access Jenkins

```text id="x5fw89"
```

http://EC2-IP:8080

```

Get password:

```bash id="nfbjlwm"

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

```

 # 5. Install Plugins

Install:

```text id="zj5zh5"

Git Plugin

GitHub Plugin

Pipeline Plugin

Docker Pipeline Plugin

```

 # 6. Create GitHub Repo

Example:

[GitHub Repo](https://github.com/Aswinvtk/aswinvtk.github.io?utm_source=chatgpt.com)

Push website files.

---

7. Create Dockerfile

```dockerfile id="pd4on0"
FROM nginx:alpine

COPY . /usr/share/nginx/html

EXPOSE 80
```

---

8. Create Jenkinsfile

```groovy id="u5m8ef"
pipeline {
  agent any

  stages {

    stage('Checkout') {
      steps {
        git 'https://github.com/Aswinvtk/aswinvtk.github.io.git'
      }
    }

    stage('Build Image') {
      steps {
```

```
            sh 'docker build -t aswinvtk/github-pages-site .'
        }
      }

      stage('Remove Old Container') {
        steps {
            sh 'docker rm -f github-pages-site || true'
        }
      }

      stage('Run Container') {
        steps {
            sh '''
            docker run -d \
            --name github-pages-site \
            -p 9090:80 \
            aswinvtk/github-pages-site
            '''
        }
      }
    }
}
```

---

9. Create Jenkins Job

```text id="mrgmje"
New Item
→ Pipeline
```

```

Settings:

```text id="2e42e7"

Pipeline script from SCM

SCM → Git

```

Repo URL:

```text id="01r84f"

https://github.com/Aswinvtk/aswinvtk.github.io.git

```

Script path:

```text id="4vgk5y"

Jenkinsfile

```

 # 10. Enable Webhook Trigger

Inside Jenkins Job:

```text id="dn2g0u"

Configure

→ Build Triggers

→ Enable:

GitHub hook trigger for GITScm polling

```

---

**# 11. Configure GitHub Webhook**

**GitHub:**

```text id="kwz0cu"
Repo
→ Settings
→ Webhooks
→ Add Webhook
```

Payload URL:

```text id="0rfvqq"
http://EC2-IP:8080/github-webhook/
```

Content type:

```text id="8q0yav"
application/json
```

Event:

```text id="m52n3w"
```
```

Just push event

```

 # 12. Push Code

```bash id="fxn7a0"

git add .

git commit -m "update"

git push origin main

```

 # 13. Auto Deployment Flow

```text id="9rxf4k"

Git Push

↓

Webhook Trigger

↓

Jenkins Build

↓

Docker Build

↓

Container Replace

↓

Website Updated

```

---

14. Useful Commands

Check containers:

```bash id="onw0gd"
docker ps
```

Check images:

```bash id="pwf6zv"
docker images
```

Check Jenkins:

```bash id="3s3x6l"
sudo systemctl status jenkins
```

Check Docker:

```bash id="yv32so"
sudo systemctl status docker
```

---

15. Website URL

```text id="t0vcmt"
http://EC2-IP:9090
```

Example:

```text id="08a0zj"
http://51.20.91.105:9090
```

Important Interview Points

CI/CD Meaning

Term	Meaning
CI	Continuous Integration
CD	Continuous Deployment

Jenkins Role

* Automates build & deployment
* Pulls latest code
* Runs pipeline

Docker Role

*** Creates portable container**

*** Same environment everywhere**

GitHub Webhook

*** Automatically triggers Jenkins after git push**

Developer Push

→ GitHub

→ Webhook

→ Jenkins

→ Docker Build

→ Container Deploy

→ Live Website